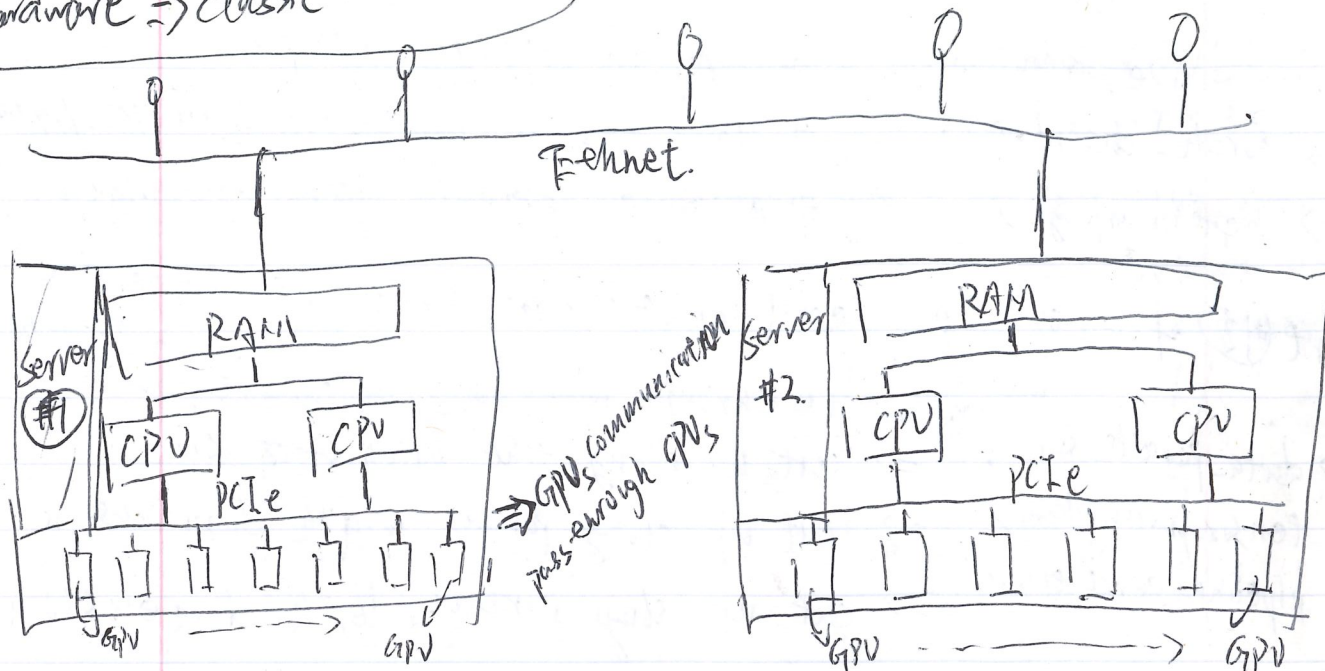


torch-distributed:

Hardware $\Rightarrow$ classic in home $\Rightarrow$ gaming / simulation



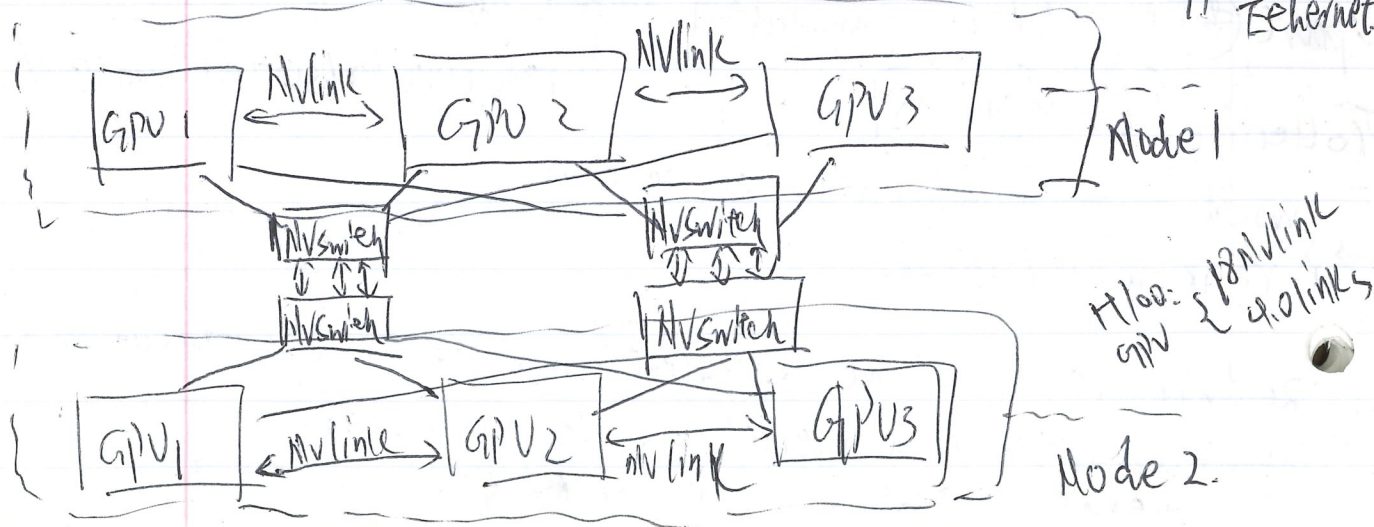
$\Rightarrow$ GPU on same node communicate via a PCIe (4: 242 GB/s)

$\Rightarrow$ GPU on different nodes communicate via Ethernet (~ 200 MB/sec)

Modern communication in data center: DL / LLM modeling

$\Rightarrow$ GPU on same node communicate via NVlink, $\rightarrow 900$ GB/s, bypass CPU

$\Rightarrow$ GPU on different nodes communicate via NVswitch, $\rightarrow 3.9$ TB/s, bypass Ethernet



How use these GPU hardware?

● Nvidia's collective communication Library (NCCL)

⇒ NCCL: translate collective operations into low-level packets that are sent between GPUs
→ (all-reduce/reduce-scatter/broadcast)

①: detects topology of hardware e.g. num of nodes, switches, NVLink/PCIe

②: optimizes the path between GPUs

③: launches CUDA kernels to send/receive data

Pytorch distributed library [torch.distributed() - ...]

↳ clean interface for collective operations

supports multiple backends for different hardware: gloo(CPU), nccl(GPU)

supports higher-level algorithms (Fullsharded Data Parallel)

def collective_operations(main_rank: int, world_size: int):

 # all-reduce dist.barrier(): waits for all processes to get to this point in case, for print screen
 # Reduce-scatter
 # All-gather

⇒ All-reduce = reduce-scatter + all-gather

input = world size
⇒ measure bandwidth | timing → [end-time - (start-time)]

def Bench-marking(): ↳ fast speed of NVLink/NVSwitch.

 # all-reduce(rank, world_size, num-elements)

 # reduce-scatter

 # all-gather:

if torch.cuda.is_available():

 torch.cuda.synchronize(): # wait for CUDA kernels to finish

 dist.barrier(): # wait for all the processes to get here